

Buenas Prácticas — SmartLogix

Cobertura, Tests Unitarios y Análisis Estático

DSY1106 Desarrollo Fullstack III — Evaluación Parcial N°2

1. Cobertura de Código (JaCoCo)

Reporte generado con `mvn verify` + `mvn jacoco:report` sobre el microservicio **Orden**.

Paquete	Instrucciones	Ramas
<code>service</code>	78%	75%
<code>controller</code>	70%	57%
<code>client</code>	60%	50%
<code>dto</code>	94%	57%
<code>security</code>	100%	100%
<code>factory</code>	100%	n/a
<code>messaging</code>	100%	n/a
Total	61%	59%

Reporte HTML completo en: <Orden/target/site/jacoco/index.html>

2. Análisis Estático (SonarCloud)

Proyecto analizado en cada push a `develop` mediante GitHub Actions + SonarCloud.

Métricas obtenidas:

- Security: **A** (0 vulnerabilidades)
 - Reliability: **A** (0 bugs)
 - Maintainability: **A** (0 code smells)
 - Duplications: **0.0%**
-

3. Ejemplos de Tests Unitarios

3.1 Test de creación de orden — `OrdenServiceTest`

Verifica que al crear una orden, el servicio obtiene el nombre real del usuario desde `UsersClient` (no usa el nombre del request), persiste la orden y publica el evento Kafka.

```
@Test
void createOrden_conUsuarioEncontrado_debeCrearOrdenConNombreDelServicio() {
    UUID userId = UUID.randomUUID();
    UUID productoId = UUID.randomUUID();

    OrdenRequestDto dto = new OrdenRequestDto();
    dto.setUserNombre("Juan Request"); // nombre que envía el frontend
    dto.setDireccionId(UUID.randomUUID());
    dto.setDetalles(List.of(detalle(productoId, 2)));

    // UsersClient devuelve el nombre real de la BD
    when(usersClient.getNombreUsuario(userId)).thenReturn("Juan García");
    when(productoClient.getProducto(productoId)).thenReturn(productoData());
    when(ordenRepository.save(any())).thenReturn(ordenGuardada(userId));
    doNothing().when(eventProducer).publishOrdenCreada(any());

    OrdenResponseDto result = ordenService.createOrden(dto, userId);

    assertNotNull(result);
    assertEquals(userId, result.getUserId());
    assertEquals("Juan García", result.getUserNombre()); // usa nombre real, no el
del request
    verify(ordenRepository, times(1)).save(any());
    verify(eventProducer, times(1)).publishOrdenCreada(any()); // evento Kafka
publicado
}
```

Por qué está bien escrito:

- Nombre descriptivo que explica el escenario y el resultado esperado
- Sigue el patrón **Arrange / Act / Assert** claramente separado
- Verifica comportamiento (interacciones con mocks) además del valor de retorno
- Prueba una sola responsabilidad del servicio

3.2 Test de validación de acceso — `OrdenServiceTest`

Verifica que el servicio rechaza correctamente cuando un cliente intenta cancelar una orden que ya fue entregada (estado inválido para cancelación).

```
@Test
void addHistorial_clienteCancelaOrdenEntregada_debeLanzarExcepcion() {
    UUID userId = UUID.randomUUID();
    OrdenModel orden = ordenSample(userId);
    orden.setEstadoActual("entregado"); // estado que impide la
cancelación

    HistorialRequestDto dto = new HistorialRequestDto();
    dto.setEstadoId(UUID.randomUUID());
    dto.setEstadoNombre("Cancelado");
}
```

```
dto.setComentario("Cancelado por el cliente");

when(ordenRepository.findById(1L)).thenReturn(Optional.of(orden));

// El servicio debe lanzar excepción
assertThrows(IllegalStateException.class,
    () -> ordenService.addHistorial(1L, dto, userId, ROL_CLIENTE));

// Nunca debe persistir el historial si la validación falla
verify(historialRepository, never()).save(any());
}
```

Por qué está bien escrito:

- Prueba el camino de error (no solo el camino feliz)
- Verifica que **no se ejecutan efectos secundarios** cuando falla la validación
- Usa `assertThrows` con el tipo de excepción exacto
- Constante `ROL_CLIENTE` en lugar de literal duplicado

3.3 Test de Circuit Breaker — `UsersClientTest`

Verifica que cuando el microservicio `Users` está caído, el Circuit Breaker activa el fallback y devuelve `null` sin lanzar excepción (el sistema sigue funcionando).

```
@Test
void getNombreUsuario_servicioUsersNoDisponible_debeRetornarNullPorFallback() {
    UUID userId = UUID.randomUUID();

    // Simulamos que el servicio externo lanza excepción (caído)
    when(restTemplate.getForObject(anyString(), eq(Map.class)))
        .thenThrow(new RuntimeException("servicio caído"));

    String resultado = usersClient.getNombreUsuario(userId);

    // El fallback devuelve null en lugar de propagar la excepción
    assertNull(resultado);
}
```

Por qué está bien escrito:

- Prueba resiliencia del sistema, no solo la lógica de negocio
- Simula una condición real de fallo de infraestructura
- Verifica que el sistema degrada graciosamente (null en vez de excepción)
- Documenta implícitamente el comportamiento del Circuit Breaker

3.4 Test del controlador con autorización — `OrdenControllerTest`

Verifica que el controlador retorna 403 cuando un cliente intenta acceder a una orden que no le pertenece.

```
@Test
void getById_accesoDenegado_retorna403() {
    MockHttpServletRequest request = requestConUserId(otroUserId);
    Authentication auth = authConRol("cliente");

    when(ordenService.getById(2L, otroUserId, "cliente"))
        .thenThrow(new RuntimeException("Acceso denegado: no es tu orden"));

    ResponseEntity<OrdenResponseDto> resp = controller.getById(2L, request, auth);

    assertEquals(HttpStatus.FORBIDDEN, resp.getStatusCode());
}
```

Por qué está bien escrito:

- Prueba la capa de presentación de forma aislada (sin levantar servidor)
- Verifica el código HTTP correcto para cada tipo de error (403 vs 404 vs 401)
- Usa `MockHttpServletRequest` para simular el contexto de seguridad

4. Configuración del Pipeline CI/CD

```
# .github/workflows/ci.yml (fragmento)
- name: Run tests with JaCoCo
  run: mvn verify -B --no-transfer-progress -s $GITHUB_WORKSPACE/settings.xml

- name: SonarCloud Analysis
  run: mvn sonar:sonar -B
      -Dsonar.projectKey=...
      -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
```

Cada push a `develop` ejecuta automáticamente:

1. Compilación con Java 21
2. 72 tests unitarios (JUnit 5 + Mockito)
3. Reporte de cobertura JaCoCo
4. Análisis estático SonarCloud

5. Resumen

Métrica	Valor
Tests unitarios totales	72
Cobertura instrucciones (local)	61%
Cobertura ramas (local)	59%

Métrica	Valor
Cobertura capa service	78%
Cobertura capa security	100%
Cobertura capa factory	100%
Bugs SonarCloud	0
Vulnerabilidades SonarCloud	0
Code smells SonarCloud	0